Initial Architecture Document

Team Number: 12
Team Members: Jaydee Brown, Aidan Bergeman, Didier A Carrillo, Will Godderz, Ethan Gao, Lea Sulthana Syed, Zackary Rhodes
Project Name: Room Manager
Project Synopsis: A cloud-hosted household manager for roommates to track shared expenses, coordinate chores, and manage grocery lists using React, C#, SQL and Render.

Architecture Description:

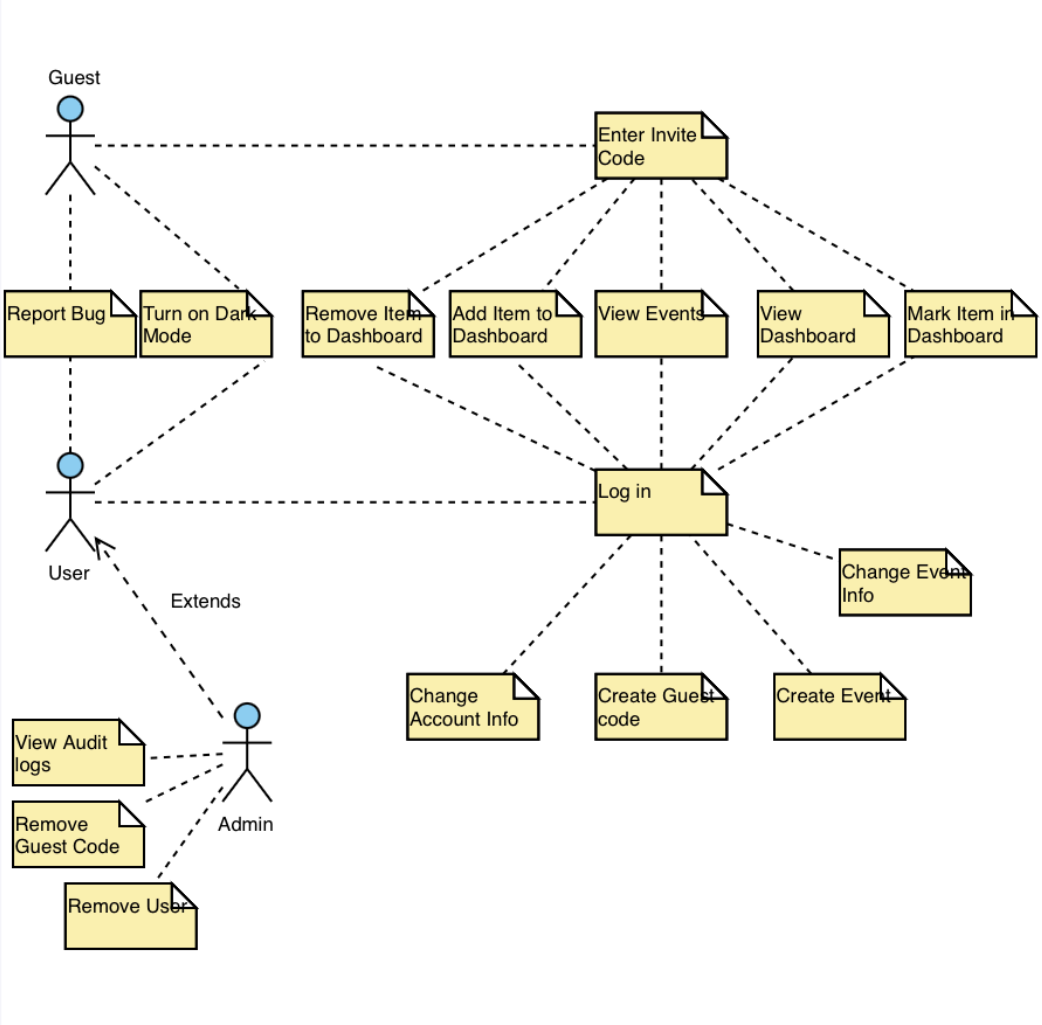
The Room Manager system architecture is engineered as a cloud-hosted, multi-tier web application designed for collaborative household living. Building upon software engineering principles and data structures mastered in EECS 168, EECS 268, EECS 348, and EECS 581 Projects 1 and 2, our team established a client-server system that balances responsive interactions with reliable cloud execution. The system coordinates three different tiers: a modern frontend user interface, a dual-service application tier combining C# and Python web services, and a persistent relational database hosted on Render.

The presentation tier is built using React, incorporating Felt UI components to provide a responsive graphical interface that works across smartphones, laptops, and desktop browsers. React manages local application state, component rendering, and user interactions for core features like dashboards, toggling chore statuses, updating grocery items, and recording shared bills. When a user interacts with the interface, React immediately updates the local visual state before dispatching HTTP network requests. Client-side input validation rejects invalid formats, empty fields, and negative monetary amounts directly in the browser to ensure bad data is never sent over the network. The compiled frontend client is deployed and served globally as a static web service hosted on Render.

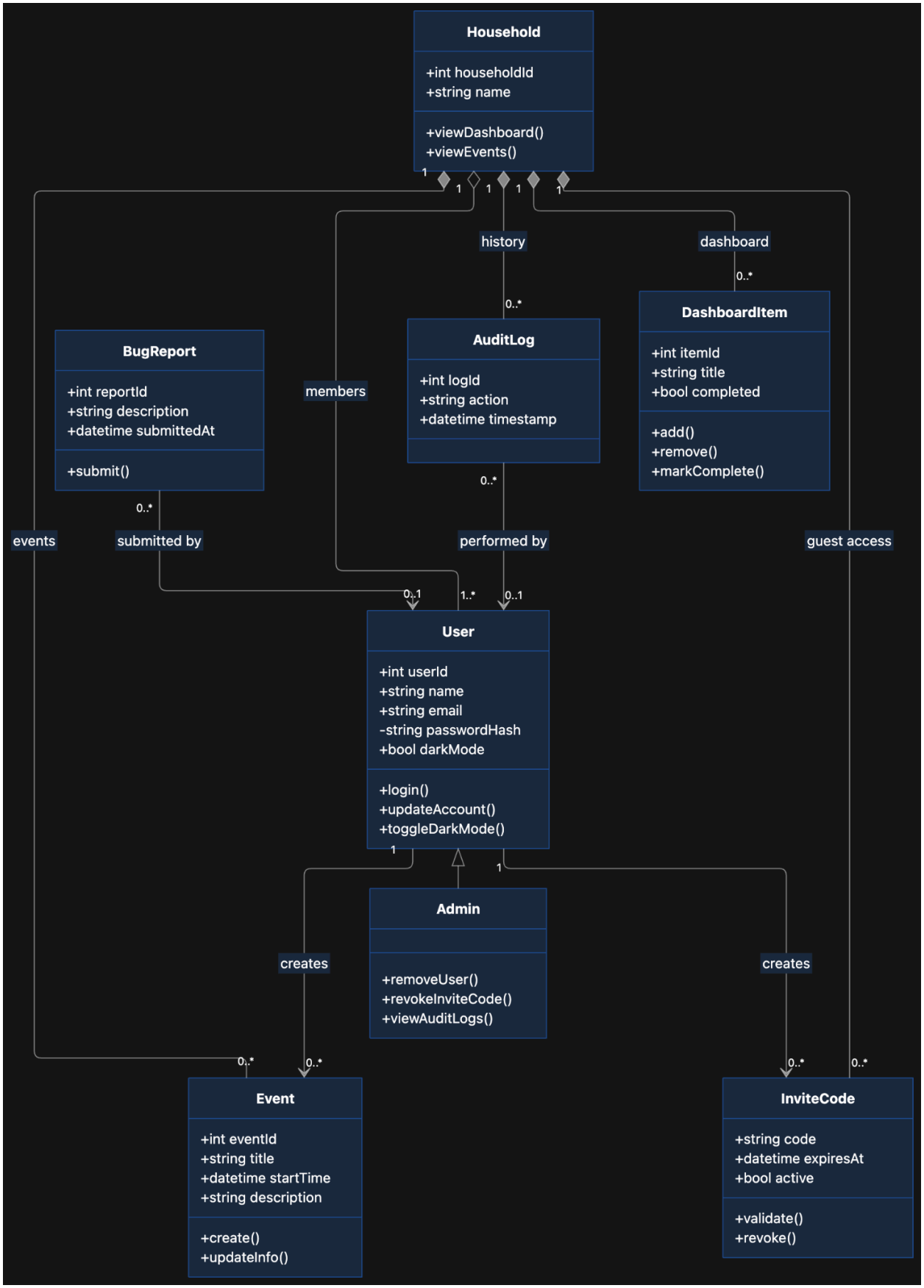
The application utilizes a hybrid backend architecture consisting of C# and Python services deployed as managed web services on Render. Core routing, user account session management, and household coordination logic are implemented in C# using ASP.NET Core. C# provides strong static typing, high performance, and structured object-oriented service layers, building upon the object-oriented design and data structure paradigms. To access backend endpoints, client requests authenticate by appending an assigned API key header or token with each call. The C# API gateway checks this incoming API key against authorized household memberships, verifying that the caller belongs to the target household before processing actions such as removing members or editing chore schedules.

To complement C# and directly built on the concepts we have learned in EECS 168, EECS 268, and EECS 348, specialized data processing tasks are routed to lightweight Python microservice endpoints hosted on Render. Rather than handling full database administrative duties, Python endpoints focus on computational operations and rapid data transformation. For example, when roommates split expenses by percentages or custom amounts, the frontend sends the values to a Python endpoint that calculates exact proportional allocations and debt matrices while preventing rounding errors. Python also provides the foundation for parsing unstructured text and future receipt data. Once computed, the formatted financial results are returned to the frontend and persisted through the C# backend to ensure centralized ledger integrity.

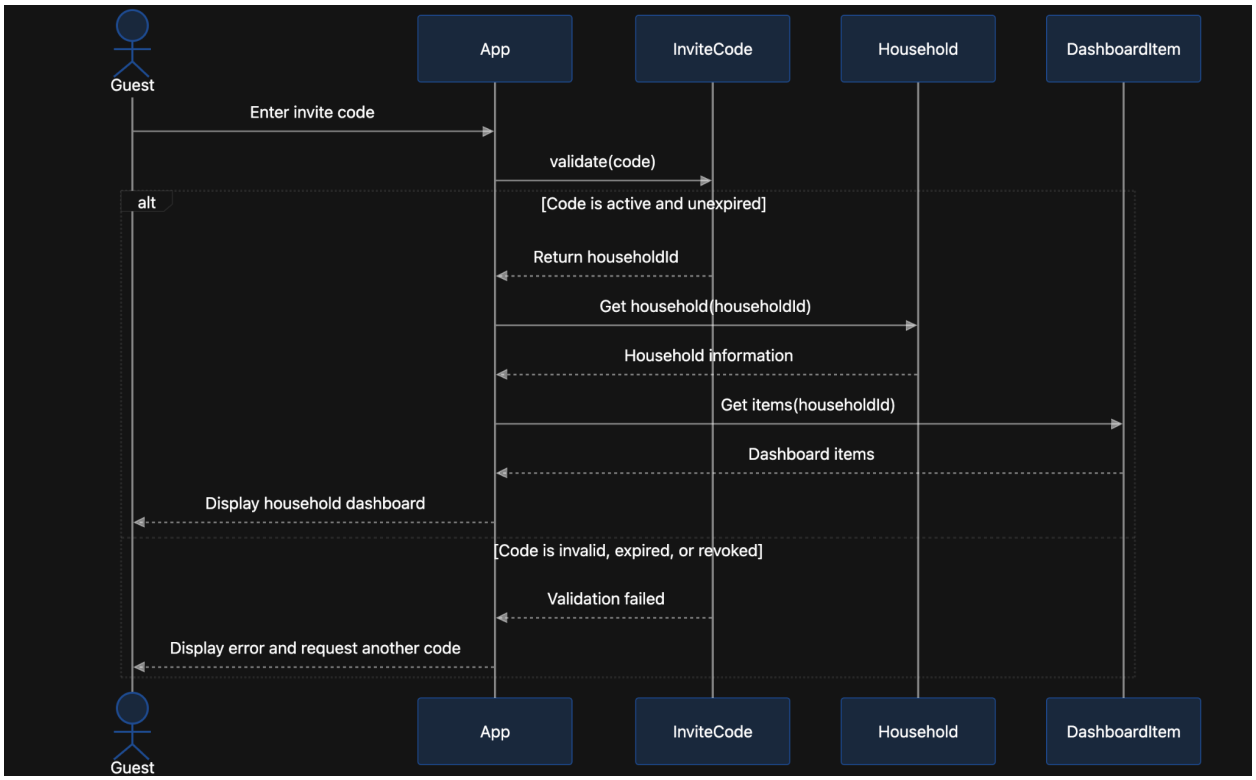
System functionality and actor permissions are defined by our Use Case model. The system recognizes three primary actors: Guest, User, and Admin. A Guest actor can enter an invite code to access household views, toggle dark mode, view calendar events, view the home dashboard, and submit bug reports. A registered User extends these privileges after logging in, gaining permissions to add/remove dashboard items, edit event schedules, and generate guest invite codes. The Admin actor inherits all User capabilities while retaining exclusive administrative rights, including viewing household audit logs, revoking guest invite codes, and removing departed roommates from the household.



The structure of Room Manager is formalized in our Class Diagram. The central Household entity maintains one-to-many relationships with members, calendar Events, and DashboardItems. Each User maintains account attributes, hashed credentials, and display preferences, with the Admin class inheriting User attributes and adding administrative actions. When a roommate creates a guest access pass, the system represents an InviteCode object containing an expiration timestamp and active status flag. Shared tasks and chores are encapsulated within DashboardItem objects, which track title, assignee, and completion status. System actions are captured as AuditLog entries, and user feedback is recorded as BugReport entities, ensuring transparent household history.


Household
+int householdId
+string name
+viewDashboard()
+viewEvents()
1
1
1
1
1
history
dashboard
0..*
0..*
DashboardItem
+int itemId
+string title
+bool completed
+add()
+remove()
+markComplete()
BugReport
+int reportId
+string description
+datetime submittedAt
+submit()
members
AuditLog
+int logId
+string action
+datetime timestamp
0..*
0..*
events
submitted by
performed by
guest access
0..1
1..*
0..1
User
+int userId
+string name
+string email
-string passwordHash
+bool darkMode
+login()
+updateAccount()
+toggleDarkMode()
1
1
Admin
+removeUser()
+revokeInviteCode()
+viewAuditLogs()
creates
creates
0..*
0..*
0..*
0..*
Event
+int eventId
+string title
+datetime startTime
+string description
+create()
+updateInfo()
InviteCode
+string code
+datetime expiresAt
+bool active
+validate()
+revoke()

The runtime behavior and object interactions are modeled in our Sequence Diagram, which illustrates the guest onboarding workflow. When an unauthenticated Guest inputs a temporary code into the React application, the client sends a validation request containing the code to the backend. The system inspects the InviteCode record to confirm that it is currently active and unexpired. If validation is successful, the backend retrieves the corresponding Household record and queries the active DashboardItems, returning the complete household view to the client. If the code is invalid/revoked, an alternative execution path returns an error message prompting the guest to enter a valid code.



Our application stores data in a PostgreSQL database hosted on Render. The database keeps information organized and ensures that related data, such as users, expenses, households, and chores stays accurate and connected. It also makes sure that data updates are completed correctly before they are saved. GitHub Actions and Render work together to automate the deployment process, so whenever an approved pull request is merged, the application is automatically built and updated with the latest version.

Other Sources & External Tools:

- Visual Paradigms: Used for creating and generating UML Use Case, Class, and Sequence diagrams.
- AI: Gemini (for formatting document)